

Documento de

Arquitectura de Software

*Sistema de Gestión de Trazabilidad
y Logística Electoral*

CNE — Delegación Provincial de Imbabura

Versión 1.0

1. Introducción

1.1 Propósito

Este documento describe la arquitectura de software del Sistema de Gestión de Trazabilidad y Logística Electoral, desarrollado para la Delegación Provincial de Imbabura del Consejo Nacional Electoral (CNE). Su objetivo es servir como guía técnica para el equipo de desarrollo durante la implementación, así como referencia para futuras tareas de mantenimiento, escalamiento y evolución del sistema.

1.2 Alcance

La arquitectura aquí descrita cubre la totalidad de las veinte (20) historias de usuario definidas en el documento de requisitos. Comprende las aplicaciones cliente (web y móvil), el backend, la capa de persistencia, los servicios de infraestructura de soporte y los servicios externos requeridos. El despliegue se contempla sobre un único entorno productivo, orquestado mediante Docker Compose.

1.3 Documentos relacionados

- Requerimientos del Sistema de Gestión de Trazabilidad y Logística Electoral — Versión 1.0.

2. Decisión arquitectónica principal

2.1 Contexto

El sistema debe soportar un máximo estimado de cien (100) usuarios concurrentes el día de las elecciones, con un dominio funcional altamente cohesivo donde la trazabilidad de operadores, kits electorales y eventos de tracking están naturalmente acoplados. El proyecto será desarrollado por un equipo reducido y desplegado en infraestructura controlada por el CNE — Delegación Provincial de Imbabura.

2.2 Decisión

Se adopta una arquitectura de **Monolito Modular** sobre el framework NestJS, con separación lógica en módulos de dominio dentro de un único proceso desplegable. Esta decisión descarta explícitamente una arquitectura de microservicios distribuidos para esta fase del proyecto.

2.3 Justificación

La decisión se fundamenta en seis (6) consideraciones técnicas y operacionales:

- Escala del sistema. El volumen estimado de 100 usuarios concurrentes es manejable por una sola instancia de backend correctamente dimensionada. Microservicios resuelven problemas de escala que este sistema no presenta.
- Orquestación con Docker Compose. Una arquitectura de microservicios productiva requiere un orquestador robusto (Kubernetes, ECS o similar), service mesh y observabilidad distribuida. Docker Compose es la herramienta idónea para un monolito acompañado de sus dependencias de infraestructura.
- NestJS provee separación modular nativa. El sistema de Modules, Providers e inyección de dependencias del framework permite alcanzar un alto grado de desacoplamiento interno equivalente al que se obtendría con servicios independientes, compartiendo a la vez el mismo proceso, la misma base de datos y transacciones ACID.
- Dominio cohesivo. Las entidades del sistema (operadores, kits, recintos, eventos de tracking, asignaciones) están fuertemente relacionadas. Particionarlas en servicios independientes obligaría a implementar transacciones distribuidas (sagas) para operaciones que en un monolito se resuelven con una transacción simple.
- Criticidad del día electoral. La jornada electoral exige máxima confiabilidad. Cada componente adicional en la arquitectura constituye un punto potencial de falla; la simplicidad operacional es una virtud no negociable en sistemas de misión crítica.
- Eficiencia para equipos reducidos. Microservicios cobran sentido organizacional cuando múltiples equipos trabajan en paralelo. Un equipo pequeño avanza significativamente más rápido sobre un monolito modular bien estructurado.

2.4 Trade-offs aceptados

La arquitectura adoptada implica las siguientes contrapartidas, consideradas asumibles para el contexto del proyecto:

- Los módulos no pueden escalar de forma independiente; la unidad de escalamiento es el monolito completo. Mitigación: el dimensionamiento del servidor cubre con holgura la carga esperada.
- Un despliegue actualiza simultáneamente todos los módulos. Mitigación: se contempla un proceso de despliegue con tiempo de inactividad mínimo (rolling restart de la instancia).
- Un error grave en un módulo puede afectar al proceso completo. Mitigación: cobertura de pruebas, manejo riguroso de excepciones y health checks con reinicio automático.

Esta arquitectura está diseñada para ser evolutiva: si en el futuro el sistema requiere escalar módulos específicos de forma independiente, NestJS permite extraer módulos como microservicios sin reescritura sustancial del código.

3. Vista lógica

La arquitectura se organiza en cuatro capas conceptuales: aplicaciones cliente, backend de aplicación, persistencia e infraestructura, y servicios externos.

3.1 Aplicaciones cliente

Aplicación web (React). Aplicación administrativa orientada a los roles de Administrador y Técnico Supervisor. Implementa los flujos de gestión (carga masiva de usuarios y militares, asignación de roles y kits, gestión de eventos electorales, generación e impresión de QR), el dashboard con KPIs en tiempo real, la consulta de bitácora y la administración de incidencias. El monitoreo en mapa también está disponible desde esta aplicación.

Aplicación móvil (React Native). Aplicación única para todos los usuarios que operan en campo. Al iniciar sesión, el sistema redirige al usuario a la pantalla inicial correspondiente a su rol (conforme al criterio HU1-CA4): el Operador de CDA accede a las funciones de registro de salida y llegada, escaneo de kits y reporte de incidencias; el Técnico Supervisor accede a la vista de monitoreo en mapa y atención de incidencias. La aplicación incluye capacidades de funcionamiento offline con sincronización en background.

3.2 Backend — Monolito modular NestJS

Una única aplicación NestJS organizada en módulos de dominio. Cada módulo encapsula sus controladores, servicios, repositorios y DTOs propios, y declara explícitamente sus dependencias con otros módulos a través del sistema de inyección de dependencias del framework. La aplicación expone dos superficies de comunicación: una API REST sobre HTTPS para las operaciones síncronas y un WebSocket Gateway (Socket.IO) para las funcionalidades en tiempo real (HU6 monitoreo de operadores y HU15 actualización del dashboard).

3.3 Persistencia e infraestructura de soporte

La capa de persistencia se compone de tres servicios complementarios, cada uno con responsabilidades específicas:

- PostgreSQL con la extensión PostGIS, como base de datos principal. PostGIS proporciona tipos de datos y funciones geoespaciales nativas (puntos GPS, cálculo de distancias, índices GiST) requeridas para el tracking de operadores (HU4, HU6) y para el filtrado geográfico del dashboard (HU15).
- Redis, como almacén en memoria para tres propósitos: cola de trabajos diferidos vía BullMQ (envío de notificaciones HU19 con reintentos, alertas programadas HU18, procesamiento por lotes de sincronización offline HU13), cache de consultas agregadas del dashboard, y control de tasa (rate limiting) de las APIs.
- MinIO, como almacenamiento de objetos compatible con S3, para los archivos binarios del sistema: fotografías cifradas del militar (HU3) y de las incidencias (HU14), así como los PDFs de QR generados para impresión (HU11). Se elige este enfoque para no sobrecargar PostgreSQL con datos binarios y para facilitar la implementación de políticas de cifrado en reposo.

3.4 Servicios externos

- Firebase Cloud Messaging (FCM), para el envío de notificaciones push a la aplicación móvil (HU19).
- Proveedor SMTP para el envío de correos electrónicos transaccionales (HU16 recuperación de contraseña, HU19 notificaciones por email). Se sugiere AWS SES o un proveedor equivalente con buena reputación de entregabilidad.

3.5 Componente de borde

Un servidor Nginx actúa como reverse proxy frente al backend, encargado de la terminación TLS, el servicio de los archivos estáticos de la aplicación web React, y el balanceo en caso de escalar horizontalmente el backend. Todo el tráfico entre las aplicaciones cliente y el backend pasa obligatoriamente por este componente sobre HTTPS (HTTP/2 cuando sea posible) y sobre WSS para los WebSockets.

4. Stack tecnológico

La siguiente tabla resume las tecnologías seleccionadas, su versión recomendada al inicio del proyecto y su propósito dentro del sistema.

Tecnología	Versión	Propósito
React	18.x	Aplicación web para administradores y técnicos supervisores.
React Native	0.74.x	Aplicación móvil única para operadores y supervisores en campo.
NestJS	10.x	Framework del backend monolítico modular.
TypeScript	5.x	Lenguaje único de toda la stack (cliente y servidor).
PostgreSQL	16.x	Base de datos relacional principal.
PostGIS	3.4.x	Extensión geoespacial de PostgreSQL para tracking GPS.
Prisma ORM	5.x	ORM para acceso a base de datos y migraciones versionadas.
Redis	7.x	Cache, sesiones, colas BullMQ y rate limiting.
BullMQ	5.x	Procesamiento de jobs en cola (notificaciones, alertas, sync).
MinIO	Última estable	Almacenamiento de objetos (fotografías cifradas, PDFs).
Socket.IO	4.x	WebSockets para tiempo real (monitoreo, dashboard).
Nginx	1.25.x	Reverse proxy, terminación TLS, servido de estáticos.
Docker + Compose	Última estable	Orquestación de todos los servicios.
Firebase FCM	API v1	Servicio externo de notificaciones push.
AWS SES / SMTP	—	Servicio externo de envío de correos electrónicos.
Pino	9.x	Logger estructurado para el backend.
Prometheus + Grafana	Última estable	Métricas y observabilidad (opcional, recomendado).

5. Módulos del backend

El backend NestJS se estructura en catorce (14) módulos de dominio, ordenados por prioridad de implementación. Este orden está diseñado para desbloquear progresivamente el flujo principal del día electoral antes que las funcionalidades de soporte y analítica.

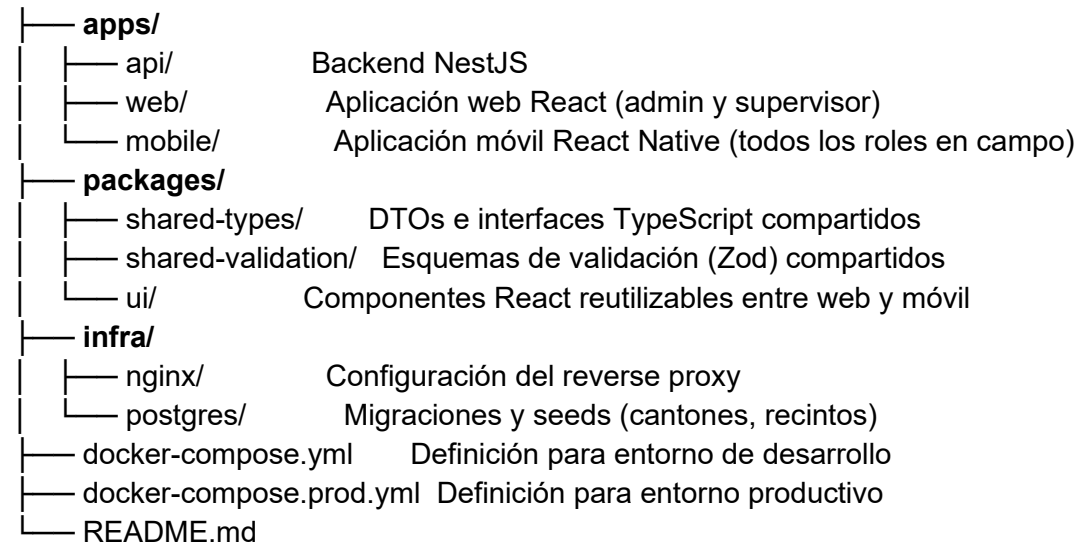
#	Módulo	Responsabilidad	HU cubiertas
1	AuthModule	Autenticación JWT, refresh tokens, hashing de contraseñas, flujo de recuperación.	HU1, HU16
2	UsersModule + RolesModule	CRUD de usuarios, carga masiva por Excel/CSV, gestión de roles y permisos (RBAC).	HU7, HU9
3	MilitaresModule + RecintosModule	Registros referenciales de militares (no operan la app) y catálogo de recintos electorales precargado.	HU8 + datos precargados
4	ElectoralEventsModule	Gestión del ciclo de vida de los eventos electorales (crear, activar, cerrar).	HU20
5	KitsModule	CRUD de kits electorales, generación de códigos QR únicos y exportación a PDF.	HU11, HU12
6	AssignmentsModule	Asignación de operadores a supervisores y de kits a recintos y operadores, con validaciones de consistencia.	HU10, HU12
7	TrackingModule	Registro de eventos de salida y llegada, recepción de puntos GPS, validación de pertenencia de kits.	HU2, HU3, HU4, HU5, HU6
8	SyncModule	Endpoint de sincronización para acciones realizadas offline, con detección de conflictos.	HU13
9	IncidentsModule	Creación, listado, seguimiento y cierre de incidencias reportadas en campo.	HU14
10	NotificationsModule	Envío de notificaciones por canal push (FCM) y	HU19

#	Módulo	Responsabilidad	HU cubiertas
		email (SMTP), preferencias por usuario, reintentos.	
11	AlertsModule	Generación de alertas automáticas mediante jobs programados en BullMQ.	HU18
12	AuditModule	Bitácora inmutable de acciones críticas, implementada mediante interceptor global de NestJS.	HU17
13	DashboardModule + ReportsModule	Agregaciones para el dashboard y exportación de reportes a CSV/PDF.	HU15
14	RealtimeGateway	Gateway WebSocket transversal para emisión de eventos en tiempo real al cliente.	HU6, HU15

6. Estructura del repositorio

Se adopta una estrategia de monorepo gestionada con pnpm workspaces. Esta organización permite compartir tipos TypeScript, esquemas de validación y componentes entre las aplicaciones cliente y el backend, garantizando consistencia y evitando duplicación de código.

cne-imbabura/



Beneficio principal: una entidad como el DTO de un kit electoral se define una sola vez en **shared-types** y se reutiliza con autocompletado y validación consistente desde las tres aplicaciones.

7. Vista de despliegue

La totalidad del sistema se despliega mediante Docker Compose sobre un único servidor (o un par de servidores en configuración activo-pasivo para alta disponibilidad opcional). La siguiente tabla enumera los servicios de Compose, su imagen base y su rol.

Servicio	Imagen base	Rol
nginx	nginx:1.25-alpine	Reverse proxy, TLS termination, sirve estáticos de la app web.
api	node:20-alpine (build propio)	Backend NestJS monolítico. Expone REST y WebSocket.
worker	node:20-alpine (build propio)	Proceso separado de NestJS que ejecuta los jobs de BullMQ (notificaciones, alertas, sincronización pesada).
postgres	postgis/postgis:16-3.4	Base de datos principal con extensión PostGIS habilitada.
pgbouncer	edoburu/pgbouncer	Pool de conexiones frente a PostgreSQL, evita saturación bajo carga del día electoral.
redis	redis:7-alpine	Cache, sesiones y broker de BullMQ.
minio	minio/minio	Almacenamiento de objetos S3-compatible para fotos y PDFs.
prometheus	prom/prometheus	(Opcional) Recolección de métricas del backend y de la infraestructura.
grafana	grafana/grafana	(Opcional) Visualización de métricas y dashboards operacionales.

7.1 Volúmenes persistentes

- `postgres_data`: datos de la base de datos.
- `redis_data`: persistencia mínima de Redis (RDB snapshots para no perder colas en reinicio).
- `minio_data`: archivos binarios almacenados.
- `nginx_certs`: certificados TLS.

7.2 Redes

- `frontend_net`: comunica nginx con api.
- `backend_net`: red interna privada que conecta api, worker, postgres, pgbouncer, redis y minio. No expone puertos al host.

7.3 Políticas de resiliencia

- Todos los servicios se declaran con `restart: unless-stopped`.
- Health checks definidos para api, postgres y redis. Docker reinicia automáticamente cualquier servicio que falle el health check.
- Los servicios dependientes esperan a que sus dependencias estén healthy antes de arrancar (`depends_on` con `condition: service_healthy`).

8. Consideraciones de seguridad

8.1 Comunicación

- Todo el tráfico entre clientes y servidor se realiza sobre HTTPS (TLS 1.2 o superior).
- Las conexiones WebSocket utilizan WSS.
- Los servicios internos (postgres, redis, minio) están aislados en la red privada de Docker y no exponen puertos al host.

8.2 Autenticación y autorización

- Autenticación basada en tokens JWT firmados, con tokens de acceso de corta duración (15 minutos) y refresh tokens de mayor duración (7 días).
- Las contraseñas se almacenan con hash bcrypt o argon2id (factor de costo apropiado).
- Control de acceso basado en roles (RBAC) implementado mediante Guards de NestJS, validados en cada endpoint.
- Política de contraseñas fuertes (mínimo 12 caracteres, complejidad mixta), conforme a HU1-CA3.

8.3 Protección de datos sensibles

- Las fotografías capturadas en campo (HU3 militar, HU14 evidencias) se cifran en reposo en MinIO usando server-side encryption (SSE-S3 o KMS).
- Las credenciales y secretos del sistema se gestionan mediante variables de entorno o un gestor de secretos (Docker secrets), nunca hardcodeadas en el código.
- Los datos personales (cédulas, emails, teléfonos) se tratan conforme a la normativa ecuatoriana de protección de datos personales.

8.4 Bitácora e integridad

- Todas las acciones críticas se registran inmutablemente en la bitácora (HU17), implementada como tabla append-only con retención mínima de cinco (5) años.
- Los códigos QR de kits electorales se validan contra la base de datos en cada escaneo (HU11-CA2) para prevenir suplantación.

8.5 Endurecimiento operacional

- Rate limiting sobre los endpoints públicos (login, recuperación de contraseña) mediante Redis.
- Validación estricta de entradas mediante DTOs con class-validator en todos los endpoints.
- Cabeceras de seguridad HTTP (Helmet) configuradas a nivel de NestJS.
- CORS restringido a los dominios oficiales de las aplicaciones cliente.

9. Consideraciones de rendimiento y resiliencia

9.1 Pool de conexiones a base de datos

Con 100 usuarios concurrentes el día electoral, abriendo conexiones para WebSockets, polling y sincronización offline, PostgreSQL puede saturarse en su límite de conexiones. PgBouncer se interpone como pool de conexiones, permitiendo que múltiples clientes compartan un conjunto reducido de conexiones reales a PostgreSQL.

9.2 Cache de consultas agregadas

Las consultas que alimentan el dashboard (HU15) son agregaciones que pueden ser computacionalmente costosas. Estas consultas se cachean en Redis con un TTL corto (5 a 15 segundos), suficiente para reducir drásticamente la carga sobre PostgreSQL sin sacrificar la sensación de tiempo real.

9.3 Procesamiento asíncrono de notificaciones

El envío de notificaciones (HU19) y la evaluación de alertas (HU18) no bloquean las solicitudes HTTP. Se encolan en BullMQ y se procesan por el servicio worker en segundo plano, con política de reintentos exponencial.

9.4 Sincronización offline robusta

La aplicación móvil almacena localmente (SQLite o WatermelonDB) todas las acciones realizadas sin conexión, junto con su timestamp y ubicación GPS. Al recuperar conexión, las acciones se envían al endpoint de sincronización en orden cronológico, preservando la integridad de la cadena de custodia.

9.5 Health checks y reinicio automático

Cada servicio del Compose declara un health check específico. Si el backend deja de responder, Docker lo reinicia automáticamente. Los clientes (web y móvil) implementan reconexión automática para WebSockets y reintento con backoff para llamadas HTTP fallidas.

9.6 Observabilidad

Los logs del backend se emiten en formato estructurado JSON mediante Pino, lo que facilita su ingesta posterior. La integración opcional con Prometheus y Grafana permite monitorear en vivo métricas clave: latencia de endpoints, tasa de errores, profundidad de colas en Redis y uso de recursos.

10. Consideraciones específicas del día electoral

La jornada electoral es el momento de máxima exigencia para el sistema. Las siguientes prácticas se incorporan al diseño para garantizar su operación confiable:

- Sobredimensionamiento controlado. El servidor productivo se dimensiona con al menos el doble de la carga estimada (CPU, RAM, conexiones), absorbiendo picos imprevistos sin degradación.
- Modo congelamiento de asignaciones. Conforme a HU12-CA6, una vez iniciado el día electoral las asignaciones quedan bloqueadas para prevenir cambios inadvertidos. Cualquier modificación excepcional requiere autorización explícita registrada en bitácora.
- Pruebas de carga previas. Antes de la jornada, se ejecutan pruebas de carga simulando el comportamiento de 100 operadores concurrentes para validar tiempos de respuesta y detectar cuellos de botella.
- Procedimiento de contingencia. Se define un plan documentado para escenarios degradados: caída del servidor, pérdida de conectividad masiva, saturación de la base de datos. Incluye un canal de comunicación alternativo (radio, telefonía) para coordinación manual si el sistema fallara.
- Backup en línea. Backup automático de PostgreSQL al inicio del día y cada dos (2) horas durante la jornada, hacia almacenamiento externo al servidor principal.
- Personal técnico de guardia. Al menos un miembro del equipo de desarrollo en disponibilidad inmediata durante toda la jornada para intervención ante incidentes.
- Modo solo lectura del histórico. Tras el cierre del evento electoral (HU20-CA6), los datos quedan en modo solo lectura, preservando la integridad de la información para futuras auditorías.

11. Próximos pasos

Con la arquitectura definida, las siguientes actividades técnicas constituyen los pasos inmediatos del proyecto:

- Diseño del modelo de datos (diagrama entidad-relación) derivado de las 20 historias de usuario.
- Definición del contrato de la API REST (especificación OpenAPI/Swagger) y del esquema de mensajes WebSocket.
- Especificación detallada del esquema offline de la app móvil y del protocolo de sincronización.
- Elaboración del docker-compose.yml inicial con todos los servicios listos para entorno de desarrollo.
- Definición de los entornos (desarrollo, staging, producción) y del pipeline de CI/CD.
- Definición formal de los requerimientos no funcionales detallados: SLA de disponibilidad, tiempos de respuesta objetivo, política de respaldo y recuperación, plan de capacitación de usuarios.
- Definición del plan de pruebas: unitarias, de integración, end-to-end y de carga.